

Programmation concurrente en Java

Jonathan Lejeune

Sorbonne Université/LIP6

SRCS – Master 1 SAR 2023/2024

sources :

Développons en Java, Jean-Michel Doudoux
Javadoc

Définitions

- **Concurrence** : Existence de plusieurs tâches de calcul pouvant s'exécuter en parallèle.
- **Parallélisme** : Le fait que plusieurs tâches puissent s'exécuter physiquement en même temps.

Parallélisme et systèmes distribués

- Un système distribué est composé de plusieurs machines possédant chacune son propre processeur et sa propre mémoire
⇒ parallélisme possible sur un réseau de machines
- Les processeurs modernes sont composés de plusieurs cœurs de calcul
⇒ parallélisme possible au sein d'une même machine

Impact sur le développement des applications réparties

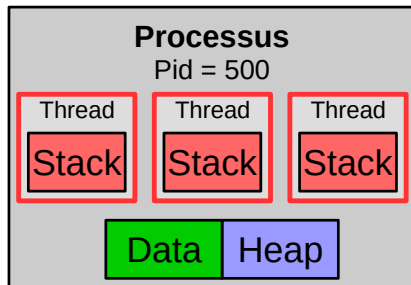
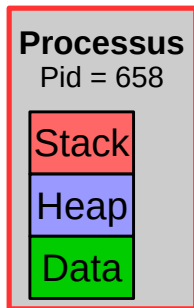
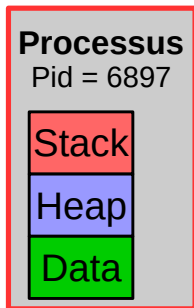
Pour tirer parti du parallélisme offert par l'infrastructure, il faut :

- concevoir l'application en la découpant en tâches concurrentes
- prendre en compte deux paradigmes de communication :
 - *Inter-machine* : passage de messages
 - *Intra-machine* : mémoire partagée
- gérer la synchronisation entre ces différentes tâches

Les limites du découpage en tâches

- **Nb de tâches concurrentes $\ll$ Nb de processeurs ou cœurs :**
 - ⇒ Séquentialisation des traitements
 - ⇒ Système surdimensionné
- **Nb de tâches concurrentes $\gg$ Nb de processeurs ou cœurs**
 - ⇒ Surcoût en mémoire
 - ⇒ Surcoût en temps de calcul (déploiement, commutation)

Exécution de tâches dans un OS



Une tâche par processus

- Meilleure isolation entre les tâches
- Communication via IPC ou I/O (pipe, socket ...)

Plusieurs tâches par processus

- Même espace d'adressage mais pile dédiée
- Crash d'une tâche peut impacter les autres
- Communication via mémoire partagée

Processus et JVM

- Une JVM est un processus du point de vue de l'OS sous-jacent
- Communications entre JVM :
 - API bas niveau : communication via I/O
 - APIs haut niveau : Java RMI, JavaEE

Thread et JVM

- Une JVM peut être subdivisée en threads Java
- Selon l'OS et l'implem. de la JVM, un thread java peut correspondre
 - soit à un thread natif de l'OS
 - soit à un thread géré exclusivement par la JVM (invisible de l'OS)
- Communications entre thread d'une même JVM :
 - Via des objets (appels de méthodes, accès aux attributs)
 - Via les variables statiques.

API native

- L'interface `Runnable`
- La classe `Thread` (ainsi que `ThreadGroup` et `ThreadLocal`)
- Les mots clés `synchronized` et `volatile`
- Les méthodes `notify`, `notifyAll` et `wait` de la classe `Object`

API du package `java.util.concurrent`

- **Planification de tâches** : `Executor`, `ExecutorService`, `Future`
- **Collections thread-safe** : `ConcurrentMap`, `BlockingQueue`...
- **Variables atomiques** : `AtomicInteger`, `AtomicReference`...
- **Synchronisation** : `Lock`, `Condition`, `ReadWriteLock`, `Semaphore`

L'interface Runnable

- Représente un objet qui définit le code d'une tâche
- N'offre qu'une seule méthode : `void run();`

La classe Thread

- Implante Runnable
- Maintient les données d'un contexte d'exécution d'une tâche
 - Le nom du thread (par défaut "Thread-*id*")
 - La référence vers le Runnable qui définit le traitement du thread
 - Un groupe auquel est rattaché le thread
 - Une priorité d'exécution
 - Un état
 - Un ClassLoader de contexte
 - Un flag indiquant si le thread est un démon ou pas
- Offre des méthodes pour interagir avec l'exécution d'une tâche

Définir une tâche

Une première solution

- Créer une classe fille qui hérite de la classe Thread.
- Redéfinir la méthode void run()
- Instancier cette classe fille

```
public MaTache extends Thread {  
    //attributs = données de la tâches  
    ...  
    //constructeurs  
    ...  
    public void run(){  
        //début des instructions de la tâche ici  
    }  
    //autres méthodes  
}
```

```
Thread t =  
    new MaTache();
```

Limite de cette approche : le lien d'héritage

- Le lien d'héritage ne peut plus être utilisé
- Fort couplage entre code métier de la tâche et contexte d'exécution

Une deuxième solution (à privilégier)

- Créer une classe (ou un lambda) qui implante l'interface Runnable.
- Définir la méthode void run()
- Instancier la classe Thread en renseignant une référence vers une instance de cette tâche

```
public MaTache implements Runnable
{
    ...
    public void run(){
        //début des instructions
    }
    ...
}
```

```
Runnable r = new MaTache();
Thread t = new Thread(r);
```

Avec un lambda :

```
Thread t = new Thread( ()-> { /*code de la tache ici */} );
```

Exécuter le code d'une tâche

Exécution dans le contexte d'exécution courant

Faire appel directement à la méthode `run` du `Runnable` ou de `Thread`.

```
Thread t = new Thread(...);  
t.run(); // NE CREE PAS DE NOUVEAU FLUX D'EXECUTION
```

Exécution dans un nouveau contexte d'exécution

- Faire appel à la méthode `start()` de la classe `Thread`
- La JVM crée une nouvelle tâche concurrente

```
Thread t = new Thread(...);  
t.start(); // Création d'un nouveau fil d'exécution
```

Arrêt d'un thread

Quand ?

- Fin de la méthode run
- Une RuntimeException est levée par la méthode run

Comment arrêter un thread depuis l'extérieur ?

- Appel à la méthode stop() :
 - Interrompt brutalement le thread en jetant une error ThreadDeath
 - **DÉPRÉCIÉ** : car ne libère pas les éventuels verrous pris par le thread
- Appel à la méthode interrupt() :
 - Positionnement d'un flag à vrai "invitant" le thread à s'arrêter
 - C'est à la responsabilité de la tâche cible d'en prendre connaissance (via isInterrupted()) et de terminer (à termes) la méthode run.
 - ⇒ permet de mettre en place un mécanisme souple d'annulation.
 - ⇒ while(true) prohibé
 - Si le thread est en attente passive :
 - on sort de la primitive bloquante par une InterruptedException
 - Le flag est repositionné à faux

Codes insensibles à `interrupt()` et solutions

Boucle infinie

```
public class MaTache implements Runnable{
    private boolean flag= true;

    public void run(){
        while(flag){
            //do something
        }
    }
}
```

Solution : utiliser `isInterrupted` dans la condition de boucle

```
public class MaTache implements Runnable{
    public void run(){
        while(! Thread.currentThread().isInterrupted()){
            //do something
        }
        //traitements de fin
    }
}
```

Codes insensibles à `interrupt()` et solutions

Mauvaise gestion de `InterruptedException`

```
public void run(){
    while(! Thread.currentThread().isInterrupted()){
        try{
            Thread.sleep(500); //on s'endort pendant 500 ms
            //do something
        } catch (InterruptedException e) {}
    }
}
```

Solution

```
public void run(){
    try{
        while(! Thread.currentThread().isInterrupted()){
            Thread.sleep(500); //on s'endort pendant 500 ms
            //do something
        }
    } catch (InterruptedException e) {...}
}
```

User thread vs. Daemon thread

Thread utilisateur

- Exécution d'une tâche classique d'une application java
- La JVM s'arrête quand **tous** les threads utilisateurs sont terminés

Thread démon

- Exécution d'une tâche de fond (ex : Garbage Collector)
- Faible priorité
- Arrêté brutalement dès l'arrêt de la JVM : les blocs `finally` ne sont pas exécutés.
⇒ **inadapté pour réaliser des opérations critiques ou I/O**

Paramétrage

Appel à la méthode `setDaemon` impérativement avant l'appel à `start()`

Autres méthodes offertes par la classe Thread

Méthodes d'instance

- `boolean` `isDaemon()`
- `long` `getId()`
- `int` `getPriority()`
- `Thread.State` `getState()`
- `ClassLoader` `getContextClassLoader()`
- `boolean` `isInterrupted()` : teste si le flag d'interruption a été positionné
- `void` `join()` OU `void` `join(long)` : attendre la terminaison d'un thread

Méthodes statiques

- `Thread` `currentThread()` : obtenir une référence sur le thread courant
- `void` `sleep(long millis)` : endormir le thread courant pendant un certain temps
- `void` `yield()` : relâcher le processeur volontairement

Ressources partagées et concurrence

```
public class Entier {  
    private int val=0; //on suppose l'existence d'un getter getVal()  
    public void incr(){  
        val++;  
    }  
}
```

Version séquentielle

```
static Entier e = new Entier();  
...  
for(int i=0;i<50;i++){  
    e.incr();  
    System.out.println(e.getVal());  
}
```

Affichage

50

Version parallèle

```
static Entier e = new Entier();  
...  
for(int i=0;i<50;i++){  
    Runnable r = ()->e.incr();  
    new Thread(r).start();  
}  
System.out.println(e.getVal());
```

Affichage

Une valeur indéterminée
comprise entre 1 et 50

Synchroniser des threads

Objectif

Définir un ordre d'exécution de tâches concurrentes sur certaines portions de code afin de garantir un état cohérent sur les données

Problèmes classiques de synchronisation

- Exclusion mutuelle
- Fork-Join
- Producteur-consommateur
- Lecteur-Écrivain

Outils de synchronisation offerts par java

- Mécanisme natif de verrouillage : Les moniteurs + blocs `synchronized`
- Depuis java5 : variables atomiques, lock, semaphores

Définition d'un verrou

Outil logiciel assurant qu'au plus une tâche concurrente puisse accéder à une portion de code appelée section critique (= **exclusion mutuelle**).

API d'un verrou

Tout verrou doit offrir a minima deux opérations **non interruptibles** :

- **Lock** $\Rightarrow$ **Entrer dans la section critique** :
 - s'assurer qu'il n'y a pas de tâche concurrente qui utilise le CS
 - s'insérer dans une file d'attente et attendre si CS non disponible
- **Unlock** $\Rightarrow$ **Sortir de la section critique** :
 - Avertir les tâches concurrentes en attente que la section critique est libre

Attention

nb d'appels d'entrée doit être = au nb d'appels de sortie

$\Rightarrow$ **Risque de famine sinon**

Retour à l'exemple

```
//instancier un verrou V
static Entier e = new Entier();
...
for(int i=0;i<50;i++){
    Runnable r = ()->{
        //lock sur verrou V
        e.incr();
        //unlock sur verrou V
    }
    new Thread(r).start();
}
//lock sur verrou V
int tmp=e.getVal();
System.out.println(tmp);
//unlock sur verrou V
```

Résultats

- À termes la valeur de e vaut bien 50
- On peut afficher une valeur comprise entre 1 et 50

Définition

Un **moniteur** est une fonctionnalité qui lie :

- un verrou
- un mécanisme de variable de condition offrant deux opérations **utilisables uniquement si le thread courant détient le verrou associé** :
 - **Wait** :
 - 1) stopper le thread et relâcher le verrou
 - 2) attendre de recevoir une notification
 - 3) reprendre le verrou
 - **Notify, NotifyAll** :
 - notifier un (ou tous les) thread(s) en attente sur la condition

Exemple

```
//instancier un moniteur M
static Entier e = new Entier();
...
for(int i=0;i<50;i++){
    Runnable r = ()->{
        //lock sur le verrou de M
        e.incr();
        //notifier sur la variable de condition de M
        //unlock sur le verrou de M
    }
    new Thread(r).start();
}
//lock sur le verrou de M
int tmp=e.getVal();
while(tmp != 50){
    //wait sur la variable de condition de M
    tmp=e.getVal()
}
System.out.println(tmp);
//lock sur le verrou de M
```

En java tout objet offre nativement les services d'un moniteur.

Le mot clé synchronized

- Mot clé associé à un bloc de code pour définir une section critique
- Les verrous Java sont réentrants
⇒ Un verrou est donné à tout thread qui le possède déjà

```
Object obj = ....  
synchronized(obj){//lock implicite  
  
    //Section critique sur obj  
  
}//unlock implicite
```

Remarques

- La prise de verrou n'est pas interruptible par un appel à `interrupt()`
- La politique d'ordonnancement sur le verrou dépend de la JVM

Les méthodes d'instance synchronized

Équivalent à prendre le verrou sur l'instance courante pour le corps de la méthode

<pre>public synchronized void f(){ //code methode }</pre>	$\Leftrightarrow$	<pre>public void f(){ synchronized(this){ //code methode } }</pre>
---	-------------------	--

Les méthodes statiques synchronized

Équivalent à prendre le verrou sur l'instance de Class pour le corps de la méthode

<pre>class A{ public static synchronized void f(){ //code methode } }</pre>	$\Leftrightarrow$	<pre>class A{ public static void f(){ synchronized(A.class){ //code methode } } }</pre>
---	-------------------	---

Variables de condition

La classe `Object` offre les méthodes suivantes :

- `void notify()` : réveiller un thread en attente sur la condition
- `void notifyAll()` : réveiller tous les threads en attente sur la condition
- `void wait()` : se mettre en attente sur la condition
- `void wait(long timeout)` : se mettre en attente sur la condition et se réveiller au plus tard dans `timeout` millisecondes

Exceptions pouvant être jetées

- `InterruptedException` : uniquement pour les méthodes `wait`
- `IllegalMonitorStateException` : si une de ces méthodes est appelée et que le thread ne possède pas le verrou correspondant.

Utilisation d'un objet dédié en tant que moniteur

```
Object m= new Object()
static Entier e = new Entier();
...
for(int i=0;i<50;i++){
    Runnable r = ()->{
        synchronized(m){
            e.incr();
            m.notify();
        };
    }
    new Thread(r).start();
}
synchronized(m){
    int tmp=e.getVal();
    while(tmp != 50){
        m.wait();
        tmp=e.getVal()
    }
    System.out.println(tmp);
}
```

Utilisation de l'objet partagé en tant que moniteur

```
static Entier e = new Entier();
...
for(int i=0;i<50;i++){
    Runnable r = ()->{
        synchronized(e){
            e.incr();
            e.notify();
        };
    }
    new Thread(r).start();
}
synchronized(e){
    int tmp=e.getVal();
    while(tmp != 50){
        e.wait();
        tmp=e.getVal()
    }
    System.out.println(tmp);
}
```

En remplaçant Entier par Integer

```
static Integer e = new Integer(0);
...
for(int i=0;i<50;i++){
    Runnable r = ()->{
        synchronized(e){
            e++;
            e.notify();
        }
    }
    new Thread(r).start();
}
synchronized(e){
    int tmp=e.getVal();
    while(tmp != 50){
        e.wait();
        tmp=e.getVal()
    }
    System.out.println(tmp);
}
```

Jette une IllegalMonitorStateException

Problème

Un bloc `synchronized` n'assure pas l'immuabilité de la référence sur lequel il s'applique :

- La variable peut être modifiée et référencer un autre objet même si un thread a posé un verrou dessus.
⇒ les appels aux méthodes `wait/notify` ne sont plus valides.
- Le moniteur change
⇒ le code n'est plus en exclusion mutuelle

Solution

- S'assurer que toutes les portions de code en exclusion mutuelle sont constamment associées au même moniteur
- Déclarer le verrou `final`.
- Ne pas poser de verrou sur des littéraux de `String`

Problème

L'utilisation d'un bloc `synchronized` impacte fortement les performances d'un programme :

- Le compilateur ne peut pas y appliquer d'optimisations
- On séquentialise l'exécution $\Rightarrow$ perte de parallélisme

Solution

Utiliser les blocs `synchronized` avec parcimonie :

- Privilégier les petites sections critiques
- Privilégier les blocs `synchronized` aux méthodes `synchronized`

Précautions à prendre

Problème

```
final Object a = new Object();  
final Object b = new Object();
```

Code thread 1

```
synchronized(a){  
    synchronized(b){  
        ...  
    }  
}
```

Code thread 2

```
synchronized(b){  
    synchronized(a){  
        ...  
    }  
}
```

⇒ **Interblocage**

Solution

Utiliser toujours le même ordre dans le cas bloc synchronized imbriqués

Avantage/inconvénient des moniteurs java

Avantages

- ✓ Facile à utiliser
- ✓ Libération automatique de la section critique garantie par le compilateur et la JVM
- ✓ Évite des bugs liés aux optimisations du compilateur ou de la JVM
exemples : changement d'ordre des instructions, utilisation de cache

Inconvénients

- ✗ Réduit les performances de l'exécution car approche bloquante
- ✗ Pas possible de vérifier l'état d'un moniteur avant de poser le verrou
- ✗ On ne peut pas différencier si on fait un accès en lecture ou en écriture des ressources partagées
- ✗ La politique d'ordonnancement du verrou (propre à la JVM) peut amener à des famines
- ✗ Chevauchement de section critique impossible

Une alternative aux moniteurs

Package `java.util.concurrent`

Interface `Lock`

- `void lock()`
- `boolean tryLock()`
- `void unlock()`
- `Condition newCondition()`

Interface `Condition`

- `void await()`
- `void signal()`
- `void signalAll()`

Classes d'implantation : `ReentrantLock`, `ReadLock`, `WriteLock`

Avantages/Inconvénients

- ✓ Meilleure souplesse sur l'acquisition et la libération du verrou
- ✗ Risque d'erreur car aucune garantie n'est faite par le compilateur pour libérer le verrou
nécessaire de faire `try{ v.lock(); /*CS*/}finally{ v.unlock(); }`
- ✗ Plus verbeux

Machine à état d'un thread Java

